

Kommander

Building Distributed Systems in **.NET**
with Embedded **Raft**



Kommander: Building Distributed Systems in .NET with Embedded Raft

**A practical guide to replicated services, partitioned
consensus, and production-grade coordination**

Andres Gutierrez

2026

Contents

1	Preface	2
1.1	Who This Book Is For	2
1.2	How to Read This Book	2
2	The Problem with One Copy	2
2.1	A Simple Service	2
2.2	The Naive Fix: Run Two Copies	4
2.3	Why “Just Copy the Data” Does Not Work	5
2.4	The Real Requirement	5
2.5	One Leader, One Order	6
2.6	Consensus	6
2.7	What Raft Provides	7
2.8	Embedded Consensus	8
2.9	What This Means for the Key/Value Service	8
2.10	Summary	9
3	Introducing Kommander	10
3.1	What Kommander Is	10
3.2	The Alternative: External Coordination	10
3.3	The Embedded Approach	11
3.4	Raft in 60 Seconds	12
3.5	What Kommander Provides vs. What You Provide	12
3.6	The Central Mental Model	13
3.7	Partitions	14
3.8	The Pluggable Architecture	15
3.9	The Entry Point: RaftManager	15
3.10	The Lifecycle	17
3.11	IRaft: The Interface	17
3.12	What This Means for the Key/Value Service	18
3.13	Summary	19

1 Preface

This book teaches you how to build distributed systems in .NET using Kommander, an open-source embedded Raft consensus library for C#.

Kommander is a library, not a database. It gives your application the machinery for multiple nodes to agree on ordered changes and recover after failures. You keep control of your API, your domain model, your state machine, and your persistence.

The book starts with a concrete problem: three instances of a service must apply the same changes in the same order without losing data when one node crashes. It builds from that problem through working code, distributed systems concepts, failure scenarios, and source-code walkthroughs.

By the end, you will be able to build, operate, and test a production-grade replicated service.

1.1 Who This Book Is For

You are a professional .NET developer who builds backend services. You are comfortable with C#, ASP.NET Core, asynchronous programming, and databases. You may know what replication means without knowing the details of Raft, quorum intersection, or log matching.

1.2 How to Read This Book

The book is organized in six parts. Parts I through III take you from zero to a partitioned replicated service. Part IV covers production operations. Part V explains the implementation internals. Part VI covers testing and verification.

You do not need to read the Raft paper first. The book introduces every concept before it depends on it.

2 The Problem with One Copy

You have a service. It stores state: customer records, account balances, session tokens, configuration entries. The service runs on one machine, backed by one process and one in-memory data structure. It works. Clients send requests, the service updates state, and everything is consistent because there is exactly one copy of the truth.

Then the machine crashes.

Every client that depended on that state is now staring at a connection timeout. If the state lived only in memory, it is gone. If it lived on disk, it is at least unavailable until the machine comes back, assuming it comes back at all. This is the oldest problem in distributed systems: a single point of failure.

The obvious fix is to run more than one copy. But the moment you do, a harder problem appears.

2.1 A Simple Service

Let's make this concrete. Here is a minimal ASP.NET Core key/value service:

```

using System.Collections.Concurrent;

var store = new ConcurrentDictionary<string, string>();

var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapPut("/kv/{key}", (string key, HttpRequest request) =>
{
    using var reader = new StreamReader(request.Body);
    string value = reader.ReadToEndAsync().Result;
    store[key] = value;
    return Results.Ok(new { key, value });
});

app.MapGet("/kv/{key}", (string key) =>
{
    return store.TryGetValue(key, out string? value)
        ? Results.Ok(new { key, value })
        : Results.NotFound();
});

app.MapGet("/kv", () =>
{
    return Results.Ok(store.ToDictionary(x => x.Key, x => x.Value));
});

app.Run();

```

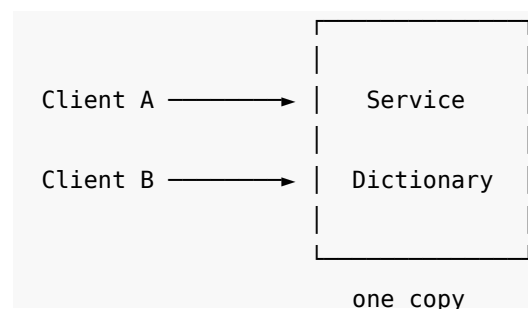
This service does exactly what it should. A client writes a value:

PUT /kv/customer:42 → body: "active"

Another client reads it:

GET /kv/customer:42 → { "key": "customer:42", "value": "active" }

The dictionary is the entire state of the system. It lives in one process, on one machine. As long as that process is running, every client sees the same state.



Now kill the process.

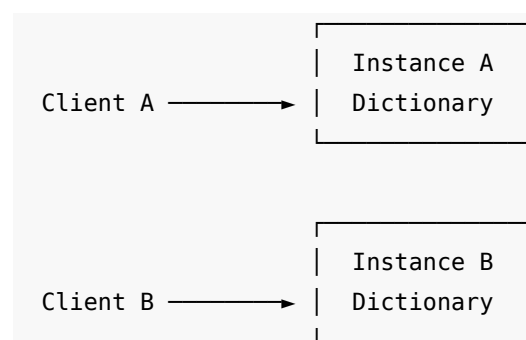
Client A → × (connection refused)

Client B → × (connection refused)

Every key is gone. Every client is blocked. The service has a single point of failure in two dimensions: it is a single point of failure for **availability** (no one can reach the data) and for **durability** (the data itself may be lost).

2.2 The Naive Fix: Run Two Copies

The instinct is to run a second instance. Maybe a load balancer distributes requests between them:



This is worse than the single-copy system. Not because it fails more often, but because it fails *silently*.

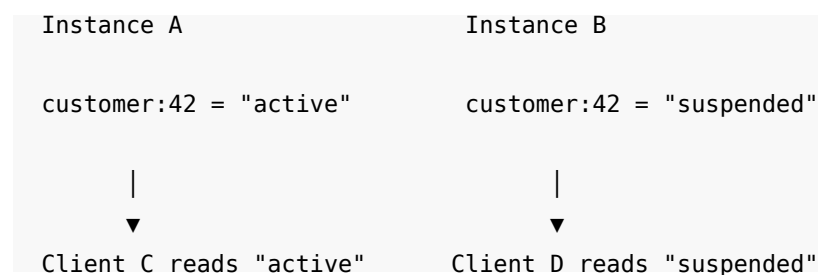
Client A writes to Instance A:

PUT /kv/customer:42 → "active" (on Instance A)

Client B writes to Instance B:

PUT /kv/customer:42 → "suspended" (on Instance B)

Now Client C reads from Instance A and sees "active". Client D reads from Instance B and sees "suspended". Neither response is wrong. Both instances faithfully served whatever was written to them. But the system as a whole is no longer consistent. Two clients asked the same question and received different answers.



This is **divergence**. The two copies of state are no longer equivalent, and no mechanism exists to reconcile them. Running two independent copies of a stateful service does not

produce redundancy. It produces two independent systems that happen to have the same API.

2.3 Why “Just Copy the Data” Does Not Work

You might think: what if Instance A forwarded every write to Instance B? Then both would have the same data.

This breaks down as soon as two writes arrive at the same time.

Consider two concurrent requests:

```
Client A → Instance A: PUT /kv/balance "100"  
Client B → Instance B: PUT /kv/balance "200"
```

Each instance applies its own write locally. Then each forwards the write to the other instance. Now both instances have received both writes. But in which order?

```
Instance A: balance = "100" → receives "200" → balance = "200"  
Instance B: balance = "200" → receives "100" → balance = "100"
```

The final state depends on the order in which the forwarded writes arrive. And that order depends on the network, which offers no guarantees. Instance A ends up with "200" and Instance B ends up with "100". The system has diverged again, despite both instances having processed the same set of writes.

This is the **ordering problem**. Two instances cannot independently decide the order of concurrent operations and arrive at the same result. The problem is not communication. Both instances received both writes. The problem is that no one decided which write goes first.

2.4 The Real Requirement

What we actually need is a system where:

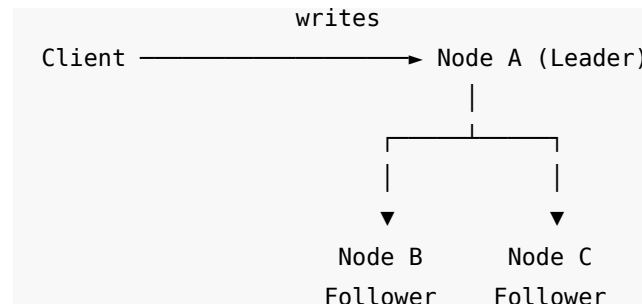
1. Multiple nodes hold copies of the same state.
2. When a client writes to any node, every node eventually reflects that write.
3. All nodes apply writes in the **same order**.
4. If a node crashes, the remaining nodes continue to serve correct state.
5. When the crashed node recovers, it catches up to the current state.

Requirements 1 and 2 describe replication. Requirement 3 is the hard one. Requirement 4 is availability. Requirement 5 is recovery.

Requirement 3, applying writes in the same order, is what distinguishes a replicated system from a collection of independent copies. Without a shared ordering, replication produces divergence. With a shared ordering, every node is a faithful replica of a single logical state machine.

2.5 One Leader, One Order

One approach to establishing a shared order: designate one node as the **leader**. All writes go through the leader. The leader assigns a sequence number to each write and sends the writes, in order, to the other nodes (the **followers**).



This is the core idea behind a family of protocols called **consensus protocols**. The leader decides the order. The followers accept it. If the leader crashes, the remaining nodes elect a new leader and continue.

But even this simple model raises questions:

- What if a follower misses a write because of a network problem?
- What if the leader crashes after sending a write to some followers but not all?
- What if two nodes both believe they are the leader?
- What if a node restarts? How does it learn what it missed?
- How do the remaining nodes know the leader has actually crashed, rather than being temporarily slow?

These are not hypothetical edge cases. They are scenarios that will occur in any system that runs long enough on real hardware and real networks. A protocol that does not handle them will corrupt data silently.

2.6 Consensus

Consensus is the formal name for the problem of getting a group of nodes to agree on a single value (or, more usefully, on an ordered sequence of values). A consensus protocol provides guarantees:

- **Agreement:** all nodes that decide on a value decide on the same value.
- **Validity:** the decided value was actually proposed by some node.
- **Termination:** every non-faulty node eventually decides.

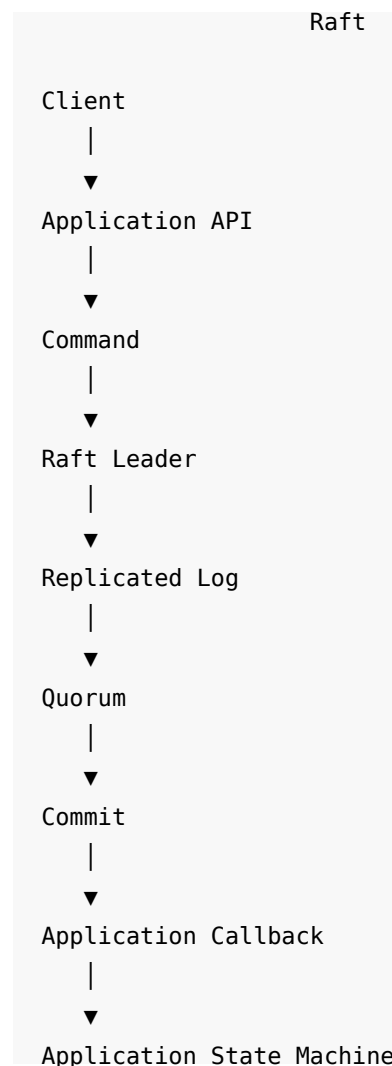
Several consensus protocols exist. Paxos is the oldest and most studied. Raft is newer, designed explicitly to be understandable. Both solve the same fundamental problem, but with different implementation approaches.

This book uses **Raft**. Not because it is better than Paxos, but because it was designed for implementors. The algorithm has a clear separation between leader election, log replication, and safety. Each piece can be understood independently.

2.7 What Raft Provides

At its core, Raft maintains a **replicated log**: an ordered sequence of entries that every node in the cluster agrees on. A leader accepts new entries, assigns them positions in the log, and replicates them to followers. An entry is **committed** once a majority of nodes have confirmed that they hold it. Once committed, the entry will not be lost even if some nodes crash.

Applications use this replicated log as a foundation. Each log entry represents a command: “set key X to value Y”, “delete key Z”, “increment counter W”. Every node processes the committed entries in the same order, and therefore every node arrives at the same state.



This separation is important. Raft replicates the *commands*. The application decides what those commands *mean*. Raft does not know anything about key/value stores, account balances, or session tokens. It guarantees that if a command is committed at position 42 in the log, then every node in the cluster will see the same command at position 42. No node will see a different command at that position.

The application takes it from there.

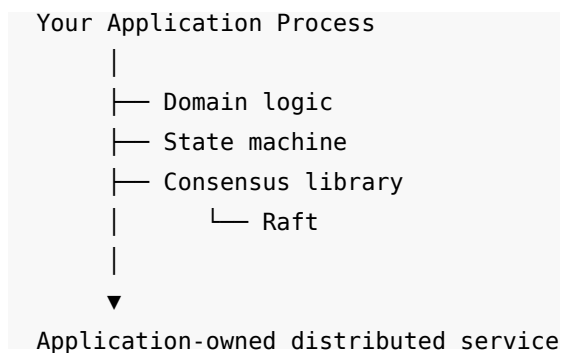
2.8 Embedded Consensus

Most developers interact with consensus indirectly. They write to etcd, or they use ZooKeeper, or they put state in a database that happens to use Raft internally. In all of these cases, the consensus protocol runs in a separate process or cluster:



This works, but it means your application depends on an external system for its most critical state. The external system must be deployed, monitored, upgraded, and secured independently. Network latency between your application and the coordination service is on the critical path of every write.

There is another approach: embed the consensus protocol directly in your application:

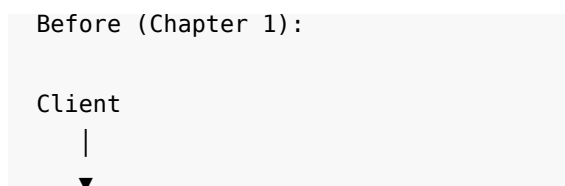


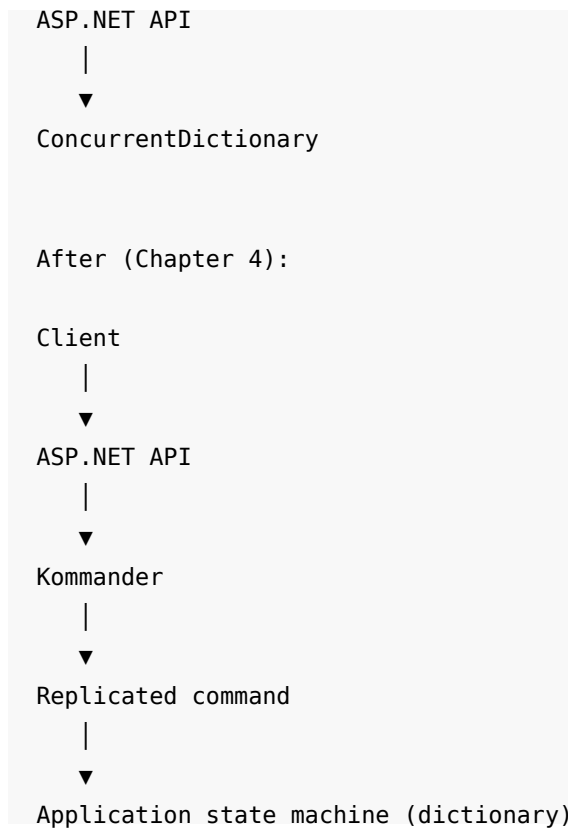
In this model, your service *is* the replicated system. There is no external dependency. Consensus runs in-process. Your application controls the API, the state machine, the persistence, and the routing. The consensus library provides the machinery for multiple nodes to agree on ordered commands.

This is the model this book teaches. The library is called **Kommander**, and the next chapter introduces it.

2.9 What This Means for the Key/Value Service

Let's revisit the simple service from the beginning of this chapter. By the end of Part I of this book, the in-memory dictionary will be replaced with a replicated state machine:





The dictionary still exists, but it is no longer the source of truth. Kommander's replicated log is the source of truth. The dictionary is a projection of the committed log, built by applying each committed command, in order, to an initially empty dictionary.

When a node crashes, the remaining nodes continue to serve reads and accept writes. When the crashed node restarts, it replays the committed log entries and rebuilds the dictionary to the current state.

That is the system we will build. The next chapter introduces the tool we will use to build it.

2.10 Summary

- A single-instance stateful service is a single point of failure for both availability and durability.
- Running multiple independent copies does not produce consistency. It produces divergence.
- Replication without ordering guarantees is insufficient. Concurrent writes applied in different orders produce different state.
- Consensus protocols solve the ordering problem by ensuring all nodes agree on the same ordered sequence of commands.
- Raft is a consensus protocol designed for implementors. It separates leader election, log replication, and safety.
- Embedded consensus places the protocol inside your application process, removing the dependency on an external coordination service.

- Kommander is a .NET library that provides embedded Raft consensus. The next chapter introduces it.

3 Introducing Kommander

Chapter 1 showed why a replicated service needs consensus. Every node must apply the same writes in the same order. A leader must coordinate that order. An election must choose a new leader when the old one fails.

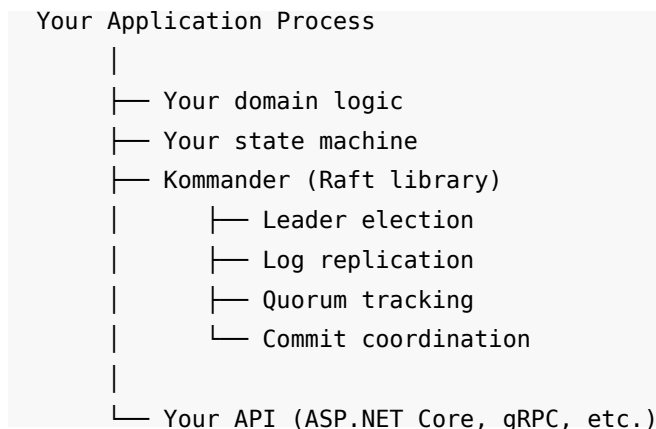
This chapter introduces the tool that provides these guarantees: **Kommander**, an open-source embedded Raft consensus library for C# and .NET.

3.1 What Kommander Is

Kommander is a library, not a database. It is not a standalone server. It is not a coordination service that you deploy and connect to. It is a NuGet package that you add to your own application:

```
dotnet add package Kommander
```

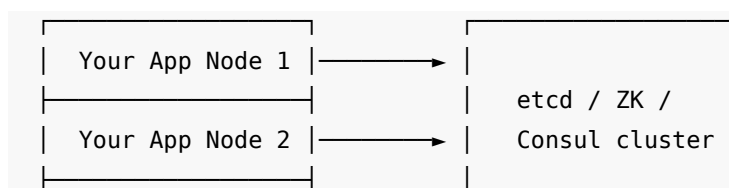
After you add it, your application process contains a Raft node. That node participates in leader election, replicates commands to other nodes, and delivers committed commands to your code through a callback. Your application controls the API, the state machine, and the domain logic. Kommander provides the replication machinery.

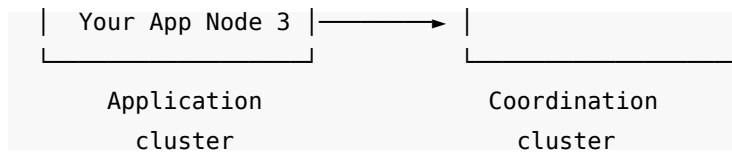


This model is called **embedded consensus**. The consensus protocol runs inside your process. There is no external service to deploy, monitor, or secure.

3.2 The Alternative: External Coordination

Most developers interact with consensus through an external coordination service. etcd, ZooKeeper, and Consul are common examples. In that model, your application connects to a separate cluster:



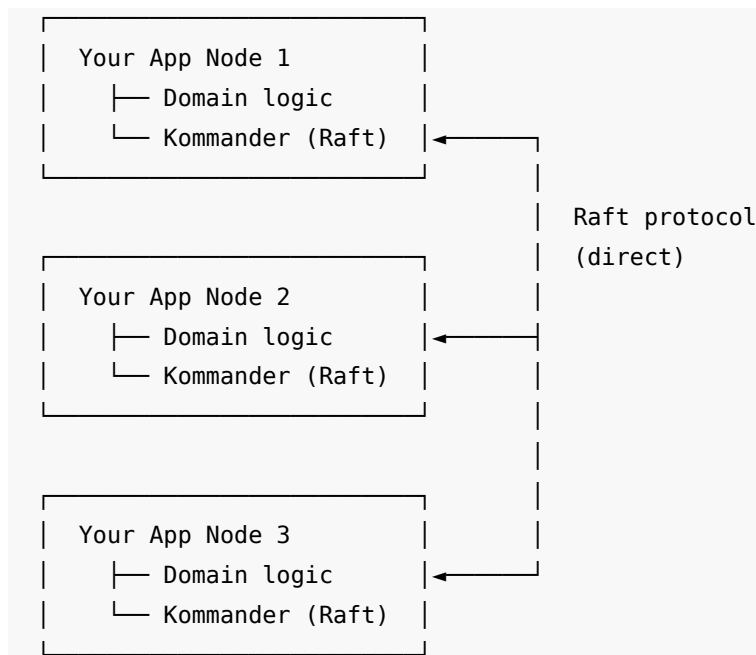


This approach works. Many production systems rely on it. But it has costs:

1. **Operational dependency.** The coordination cluster must be deployed, upgraded, and monitored independently from your application. If it goes down, your application cannot coordinate.
2. **Network latency.** Every write passes through the network to the coordination service and back. That round trip sits on the critical path.
3. **Schema mismatch.** The coordination service has its own data model (key/value, znodes, service entries). You must translate between your domain and the service's model.
4. **Capacity limits.** The coordination service has its own throughput limits. Your application must share them with all other clients of that service.

3.3 The Embedded Approach

With embedded consensus, each application node runs a Raft node inside its own process:



The Raft nodes communicate directly with each other. There is no external service. Each application process is both a Raft participant and a domain service.

This approach has its own trade-offs:

1. **Simplicity.** One deployment, not two. No separate cluster to manage.
2. **Latency.** Consensus happens in-process and over direct node-to-node links. No extra hop.

3. **Responsibility.** Your application now owns the consensus layer. You configure it, tune it, and reason about it.

Kommander is designed for the third point. It exposes the consensus primitives so that you can build on them, while it handles the protocol correctness internally.

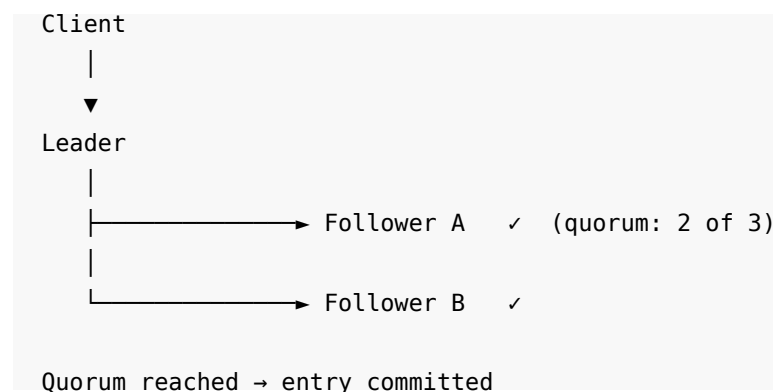
3.4 Raft in 60 Seconds

Raft is the consensus protocol that Kommander implements. Here is the essential model.

A **cluster** is a set of nodes. Each node is in one of three states for a given partition: **leader**, **follower**, or **candidate**.

- The **leader** accepts commands from clients. It assigns each command a position in an ordered log. It sends the log entries to the followers.
- The **followers** receive log entries from the leader and write them to their own logs.
- A **candidate** is a node that is running an election to become leader.

The key guarantee: a log entry is **committed** when a majority of nodes (a **quorum**) have confirmed that they hold it. Once committed, the entry will not be lost, even if some nodes crash.



When the leader crashes, the followers notice that heartbeats have stopped. One of them becomes a candidate, requests votes, and wins an election. The new leader picks up where the old one left off.

This is a simplified view. The full Raft protocol handles many edge cases: split votes, log divergence, network partitions, and more. Later chapters cover these details. For now, the important point is: Raft gives you an ordered, replicated, durable log. Your application builds its state on top of that log.

3.5 What Kommander Provides vs. What You Provide

The boundary between Kommander and your application is clear:

Kommander provides	Your application provides
Leader election	API endpoints (HTTP, gRPC, etc.)
Log replication and quorum	Domain logic and validation
Commit coordination	State machine (the thing that processes commands)
Heartbeats and failure detection	Command serialization (encoding commands as bytes)
Log persistence (WAL adapters)	Request routing (sending writes to the leader)
Cluster membership changes	Read strategy (local reads vs. confirmed reads)

Kommander does not know what your commands mean. It treats every command as a byte array with a string type label. It guarantees that if a command is committed at log position 42, every node in the cluster will see the same command at position 42. Your application decides what that command does.

Kommander's view:

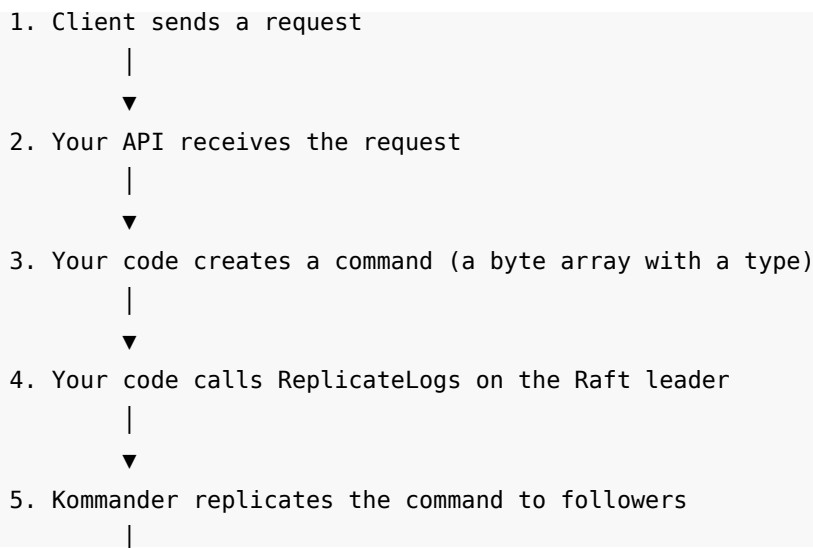
```
Log position 42:  type = "SetValue"
                  data = [0x7B, 0x22, 0x6B, ...]    (opaque bytes)
```

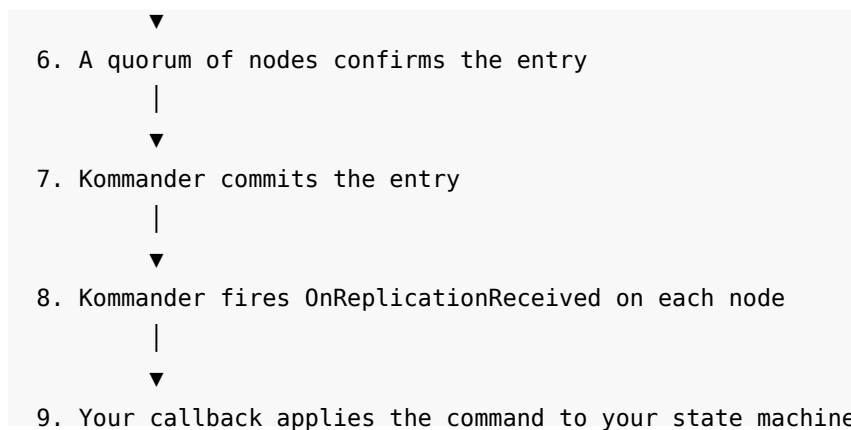
Your application's view:

```
Log position 42:  {"key": "customer:42", "value": "active"}
                  → dictionary["customer:42"] = "active"
```

3.6 The Central Mental Model

Every write in a Kommander-based application follows the same flow:





Steps 4 through 8 are Kommander’s responsibility. Steps 1 through 3 and step 9 are yours.

This flow is the same whether you have one node or fifty. It is the same whether you use an in-memory WAL or RocksDB. It is the same whether nodes communicate over gRPC, REST, or in-memory channels.

3.7 Partitions

A single Raft group processes commands sequentially through one leader. For many applications, one group is enough. But when throughput demands grow, a single leader becomes a bottleneck.

Kommander supports **partitioned Raft**. Instead of one Raft group for all commands, you can run multiple independent groups called **partitions**. Each partition elects its own leader, maintains its own log, and operates independently.

Node A	Node B	Node C
Partition 1: Leader	Partition 1: Follower	Partition 1: Follower
Partition 2: Follower	Partition 2: Leader	Partition 2: Follower
Partition 3: Follower	Partition 3: Follower	Partition 3: Leader

A node can be leader for some partitions and follower for others. This distributes the write load across the cluster.

3.7.1 Partition 0: The System Partition

Kommander reserves **partition 0** as the **system partition**. This partition stores the cluster’s own configuration: the partition map, membership roster, and other internal state. Kommander manages it automatically. Your application data uses partitions numbered 1 and above.

The `InitialPartitions` setting in `RaftConfiguration` controls how many application partitions the cluster creates at startup. The default is 1.

Partitions are a scalability feature. Part III of this book covers them in detail. For now, the examples use a single application partition (partition 1).

3.8 The Pluggable Architecture

Kommander separates the consensus core from the infrastructure it depends on. Four interfaces define that boundary:

1. **Storage (IWAL).** The write-ahead log. Kommander ships adapters for RocksDB, SQLite, and in-memory storage. RocksDB is the production choice. In-memory is useful for tests.
2. **Communication (ICommunication).** The network transport. Kommander ships gRPC and REST adapters, plus an in-memory adapter for tests. gRPC is the production choice.
3. **Discovery (IDiscovery).** How a node finds its peers. Static discovery uses a fixed list of endpoints. Dynamic discovery lets you update the list at runtime. Multicast discovery finds peers on the local network.
4. **Time (HybridLogicalClock).** A hybrid clock that combines wall-clock time with a logical counter. Kommander uses it for proposal tickets and causality tracking.

You choose the implementations that match your environment:

Concern	Development/Test	Production
Storage	InMemoryWAL	RocksDbWAL or SqliteWAL
Communication	InMemoryCommunication	GrpcCommunication
Discovery	StaticDiscovery	StaticDiscovery or DynamicDiscovery
Clock	HybridLogicalClock	HybridLogicalClock

This pluggability is not just a test convenience. It lets you evolve your infrastructure choices without changing your application logic. The consensus behavior stays the same regardless of which adapters you plug in.

3.9 The Entry Point: RaftManager

`RaftManager` is the class that brings everything together. It implements the `IRaft` interface, which defines all the operations you need: joining a cluster, checking leadership, replicating commands, and leaving the cluster.

Here is how you construct a `RaftManager`:

```
using Kommander;  
using Kommander.Communication.Grpc;  
using Kommander.Discovery;
```

```

using Kommander.Time;
using Kommander.WAL;
using Microsoft.Extensions.Logging;

ILoggerFactory loggerFactory = LoggerFactory.Create(
    builder => builder.AddConsole()
);

ILogger<IRaft> logger = loggerFactory.CreateLogger<IRaft>();

RaftConfiguration configuration = new()
{
    NodeName = "node-1",
    Host = "localhost",
    Port = 8001,
    InitialPartitions = 1
};

IRaft raft = new RaftManager(
    configuration,
    new StaticDiscovery([
        new RaftNode("localhost:8002"),
        new RaftNode("localhost:8003")
    ]),
    new RocksDbWAL(
        path: "./data",
        revision: "node-1",
        logger
    ),
    new GrpcCommunication(),
    new HybridLogicalClock(),
    logger
);

```

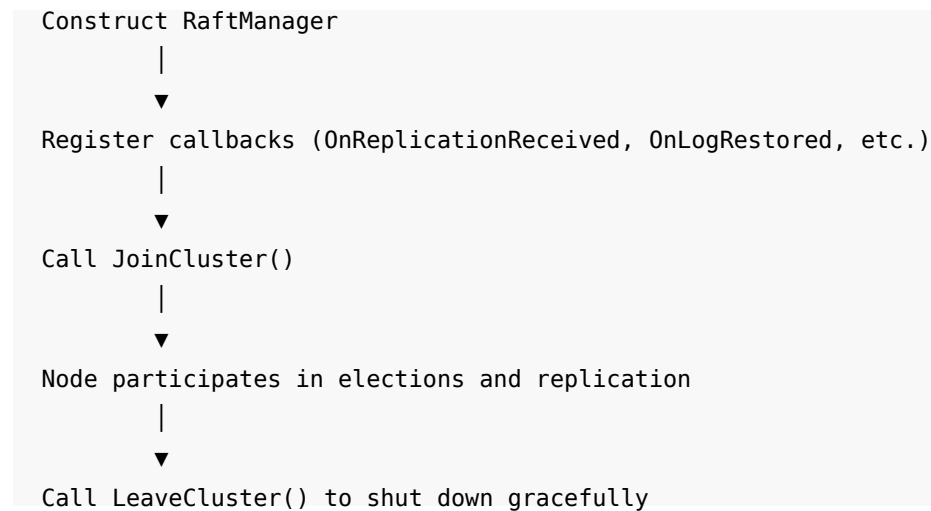
The constructor takes six arguments:

1. **RaftConfiguration** sets the node identity (name, host, port) and cluster settings (initial partitions, timeouts, intervals). Chapter 3 covers the essential properties. Appendix C is the full reference.
2. **IDiscovery** tells the node where to find its peers.
3. **IWAL** is the write-ahead log that persists Raft entries.
4. **ICommunication** is the network transport for Raft messages.
5. **HybridLogicalClock** provides timestamps for proposals.
6. **ILogger** receives diagnostic log output.

The `RaftManager` does not start any cluster activity at construction time. It only begins participating in the cluster when you call `JoinCluster`.

3.10 The Lifecycle

A Kommander node goes through a simple lifecycle:



After `JoinCluster`, the node discovers its peers, participates in leader elections, and begins accepting or forwarding commands. After `LeaveCluster`, the node stops participating and optionally disposes its resources.

The two critical callbacks are:

- **OnReplicationReceived:** fires when a committed command is ready for your application to apply. This is where you update your state machine.
- **OnLogRestored:** fires during startup for each committed entry that the WAL already contains. This is how your application rebuilds its state after a restart.

3.11 IRaft: The Interface

`IRaft` is the interface that `RaftManager` implements. It defines the full surface area that your application uses. Here are the methods and events that matter most in Part I:

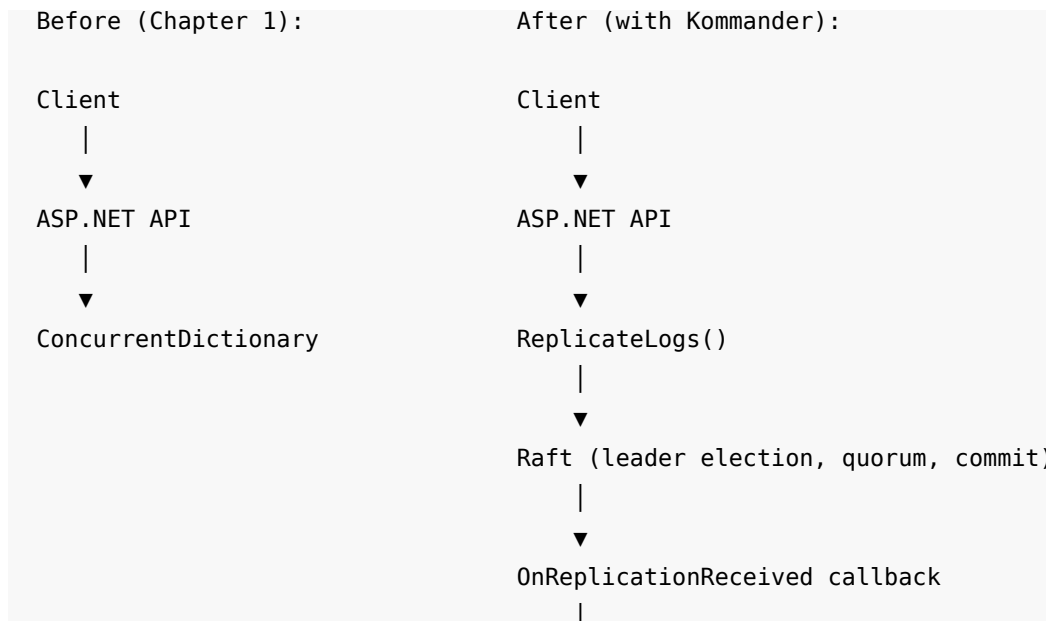
Member	Purpose
<code>JoinCluster()</code>	Start the node and join the cluster
<code>LeaveCluster(dispose)</code>	Stop the node and leave the cluster
<code>AmILeader(partitionId, ct)</code>	Check if this node leads the given partition
<code>ReplicateLogs(partitionId, data)</code>	Propose a command for replication
<code>OnReplicationReceived</code>	Callback: a committed command is ready to apply
<code>OnLogRestored</code>	Callback: a persisted command is being replayed on startup
<code>OnLeaderChanged</code>	Callback: a new leader was elected for a partition

The remaining members of `IRaft` handle advanced features: batch replication, manual commit/rollback, partition management, read confirmation, and cluster membership. Later chapters introduce them as needed.

3.12 What This Means for the Key/Value Service

In Chapter 1, the key/value service used a `ConcurrentDictionary` as its only state. The dictionary was the source of truth. When the process crashed, the state was gone.

With Kommander, the dictionary still exists. But it is no longer the source of truth. The Raft log is the source of truth. The dictionary is a local projection built by applying each committed command in order.



▼

Dictionary (local projection)

When a node crashes, the remaining nodes continue to serve reads and accept writes. When the crashed node restarts, Kommander replays the WAL entries through the `OnLogRestored` callback. The application rebuilds the dictionary from those entries.

The next chapter builds this system step by step.

3.13 Summary

- Kommander is a .NET library that embeds Raft consensus in your application process.
- The embedded approach removes the dependency on an external coordination service.
- Raft guarantees an ordered, replicated, durable log. Your application builds state on that log.
- Kommander handles leader election, log replication, quorum, and commit. Your application handles the API, domain logic, and state machine.
- `RaftManager` is the entry point. It takes a configuration, discovery, WAL, communication, clock, and logger.
- Four pluggable interfaces (`IWAL`, `ICommunication`, `IDiscovery`, `HybridLogicalClock`) let you choose the right implementation for your environment.
- Partition 0 is the system partition. Application data uses partitions numbered 1 and above.
- The next chapter creates a single-node instance and replicates the first command.